

CSE 432 Robotics

Advanced Topics in Rigid-Body Transformations and 3D Rotations

Ahmed Asker, Ph.D.

Associate Professor in Mechatronic and Robotics

Ahmed.asker@ejust.edu.eg

Egypt-Japan University of Science and Technology



Linear Transformation

- A matrix A does not just store numbers it **defines a linear transformation** that maps vectors to new vectors.
- For any 2×2 matrix

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \mathbf{v} = \begin{bmatrix} x \\ y \end{bmatrix}, \rightarrow A\mathbf{v} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix}$$

- **The vector $A\mathbf{v}$ is a new vector in $\mathbb{R}^2$ corresponding to a geometric operation such as:**
 - **rotation**
 - **scaling**
 - **shearing**
 - **reflection**
 - or any combination of these



Linear Transformation

- Let the unit square (centered at the origin) have vertices:
 $\mathbf{v}_i \in \{(-1, -1), (1, -1), (1, 1), (-1, 1)\}.$

- Choose a linear transformation

$$A = \begin{bmatrix} 1.2 & 0.6 \\ -0.3 & 1.1 \end{bmatrix}.$$

File: [Gen_Lin_Trans.m](#)

- This A combines **scale, shear, and a small rotation**.
- Area scaling factor = $\det(A) = 1.5$.
- The transformed square's area is $1.5 \times$ the original.
- Each vertex maps as $\mathbf{v}'_i = A\mathbf{v}_i$. For example:
 $(-1, -1) \mapsto (-1.8, -0.8), \quad (1, -1) \mapsto (0.6, -1.4)$
 $(1, 1) \mapsto (1.8, 0.8), \quad (-1, 1) \mapsto (-0.6, 1.4)$



Linear Transformation

- A linear transformation always preserves:
 - the origin mapped to the origin,
 - straight lines mapped to straight lines,
 - parallel lines remaining parallel.
- Therefore, matrix multiplication is not just arithmetic it is a **geometric operator** acting on vectors.

File: Mat_trans.m



Linear Transformation

- All transformations defined in 2D have direct analogues in 3D.
 - **2D rotation** → rotation about the **z-axis**, or any axis in 3D
 - **2D scaling** → independent scaling along **x, y, z**
 - **2D shear** → shear in any plane (e.g., xy , xz , yz)
 - **2D reflection** → reflection about any 3D plane
- In 3D, transformations are represented by a “ 3×3 matrix” for linear transformations (rotation, scaling, shear, reflection)
- Only 2D transformation is treated here for simplicity.



Linear Transformations in 2D

- **Rotation by θ (counterclockwise):**

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix},$$
$$v' = R(\theta) v$$

- Preserves **lengths** and **angles**; rotates directions by θ .
- $R(\theta)$ is **orthogonal** ($R^\top R = I$), $\det R = +1$, inverse is $R(-\theta) = R^\top$.



Linear Transformations in 2D

- Linear transformations is performed by a 2×2 matrix A , which act on a vector

$$v = \begin{bmatrix} x \\ y \end{bmatrix} \in \mathbb{R}^2$$

Pure linear map (no translation)

$$v' = A v$$

- All linear transformations **fix the origin** $(0, 0)$ and preserve straight lines.



Linear Transformations in 2D

- **Uniform Scaling**

$$S(s) = s I = \begin{bmatrix} s & 0 \\ 0 & s \end{bmatrix},$$
$$v' = S(s) v$$

- Multiplies **lengths** by s , **preserves angles** and **ratios of lengths** along any direction.
- $\det S = s^2$ (area scales by s^2)



Linear Transformations in 2D

- **Non-Uniform Scaling**

$$S(s_x, s_y) = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix},$$
$$v' = S(s_x, s_y) v.$$

- Stretches differently along x and y axes; **angles not preserved** unless $s_x = s_y$.
- $\det S = s_x s_y$



Linear Transformations in 2D

- **Reflection (Flip)**

- **Across the x -axis:**

$$R_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, (x, y) \mapsto (x, -y), \det R_x = -1.$$

- **Across the y -axis:**

$$R_y = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}, (x, y) \mapsto (-x, y), \det R_y = -1.$$

- **Across the origin (point inversion):**

$$R_0 = -I = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}, (x, y) \mapsto (-x, -y), \det R_0 = +1.$$

- **Notes:** Reflections have orthonormal rows/columns, are **involutive** ($R^2 = I$), and flip orientation if $\det = -1$.



Linear Transformations in 2D

- **Shear (Skew / Distortion)**

- **Horizontal shear (slide along x by $k_x \cdot y$):**

$$H_x(k_x) = \begin{bmatrix} 1 & k_x \\ 0 & 1 \end{bmatrix}, (x, y) \mapsto (x + k_x y, y).$$

- **Vertical shear (slide along y by $k_y \cdot x$):**

$$H_y(k_y) = \begin{bmatrix} 1 & 0 \\ k_y & 1 \end{bmatrix}, (x, y) \mapsto (x, k_y x + y).$$

- Preserves **area** ($\det = 1$) and parallelism but not angles or lengths;



Linear Transformations in 2D

- **Identity and Projections**

- **Identity (no change):**

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, v' = v, \det I = 1.$$

- **Orthogonal projection onto the x -axis:**

$$P_x = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, (x, y) \mapsto (x, 0).$$

- **Orthogonal projection onto the y -axis:**

$$P_y = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}, (x, y) \mapsto (0, y).$$

- Projections are **idempotent** ($P^2 = P$), have $\det = 0$ (they collapse area to a line).



Numerical Drift in Successive 3D Rotations

- A **true rotation matrix** R must satisfy:
$$R^T R = I, \det(R) = 1.$$
- These conditions guarantee that the transformation is a **pure rotation** with:
 - no scaling
 - no shear
 - orthonormal coordinate axes
 - preserved vector lengths and angles



Numerical Drift in Successive 3D Rotations

- In practical computation (e.g., simulation, robotics, IMU integration), a rotation matrix is often updated repeatedly:

$$R_{k+1} = R_k R_{\Delta}$$

- where R_{Δ} is a small rotation step (often from Euler angles or angular velocity).
- Because of **floating-point rounding errors**, after many iterations:

$$R_k^{\top} R_k \neq I, \det(R_k) \neq 1.$$



Numerical Drift in Successive 3D Rotations

- The matrix R_k is no longer orthogonal.
It begins to represent a more general **linear transformation**:

$$\mathbf{v}' = R_k \mathbf{v}$$

- that may unintentionally include:
 - **shear**
 - **anisotropic scaling**
 - **axis non-orthogonality**
 - **volume distortion** ($\det \neq 1$)



Numerical Drift in Successive 3D Rotations

- What began as a mathematically pure rotation becomes, in floating-point arithmetic:

$$R_k = \text{Rotation} + \text{Shear} + \text{Scale} + \text{Drift}.$$

- This is why long chains of rotation-matrix updates must be periodically **re-orthonormalized** (e.g., via SVD or Gram–Schmidt) or replaced with more stable representations such as **unit quaternions**.



Example: Drift Under Successive Rotations

- **Update rule (single (float) precision).**

$$R_{k+1} = R_k R_{\text{step}}, R_{\text{step}} = R_z(\alpha) R_y(\beta) R_z(\gamma).$$

- **Rotation properties (ideal).**

$$R^T R = I, \det(R) = 1.$$

- **Determinant drift:** $\det(R_k)$ deviates from 1 $\rightarrow$ **volume scaling** appears.



Example: Drift Under Successive Rotations

- **Orthogonality error** (Frobenius norm):

$$\| R_k^\top R_k - I \|_F = \sqrt{\sum_{i,j} (R_k^\top R_k - I)_{ij}^2}$$

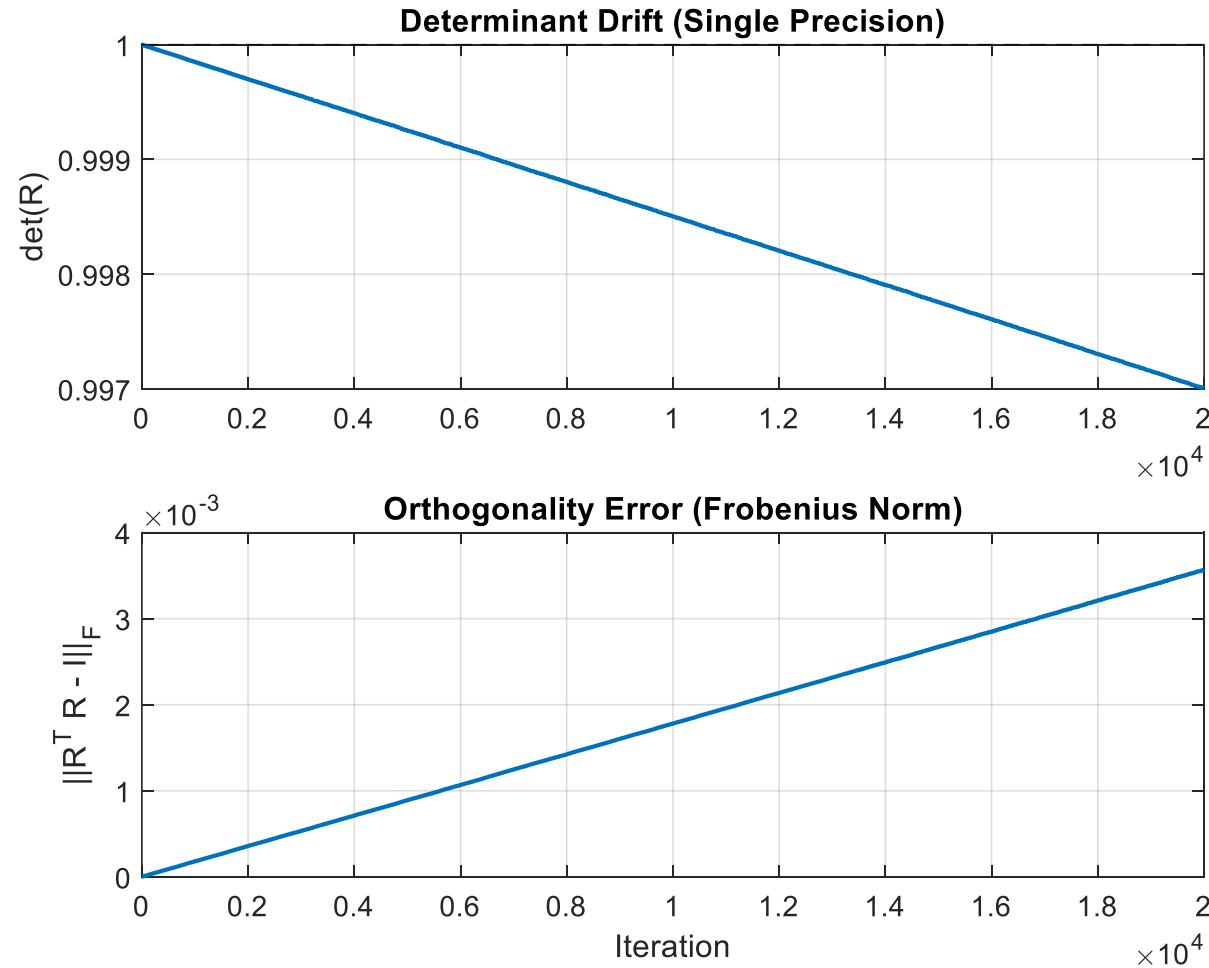
- grows as columns lose unit length and perpendicularity → **shear/scale** creep in.
- With thousands of steps in **single precision**, both metrics drift:

$$\begin{aligned} \det(R_k) &\neq 1 \\ \| R_k^\top R_k - I \|_F &> 0 \end{aligned}$$



Example: Drift Under Successive Rotations

Rot_Drift.m





Example: Orthogonality Drift

- **Update rule (single precision).**

$$R_{k+1} = R_k R_{\text{step}}, R_{\text{step}} = R_z(\alpha) R_y(\beta) R_z(\gamma).$$

- **Rotation properties (ideal).**

$$R^T R = I, \det(R) = 1.$$

- **What we measure.**

- **Determinant drift:** $\det(R_k)$ deviates from 1 $\rightarrow$ **volume scaling** appears.



Example: Orthogonality Drift

- **Orthogonality error** (Frobenius norm):

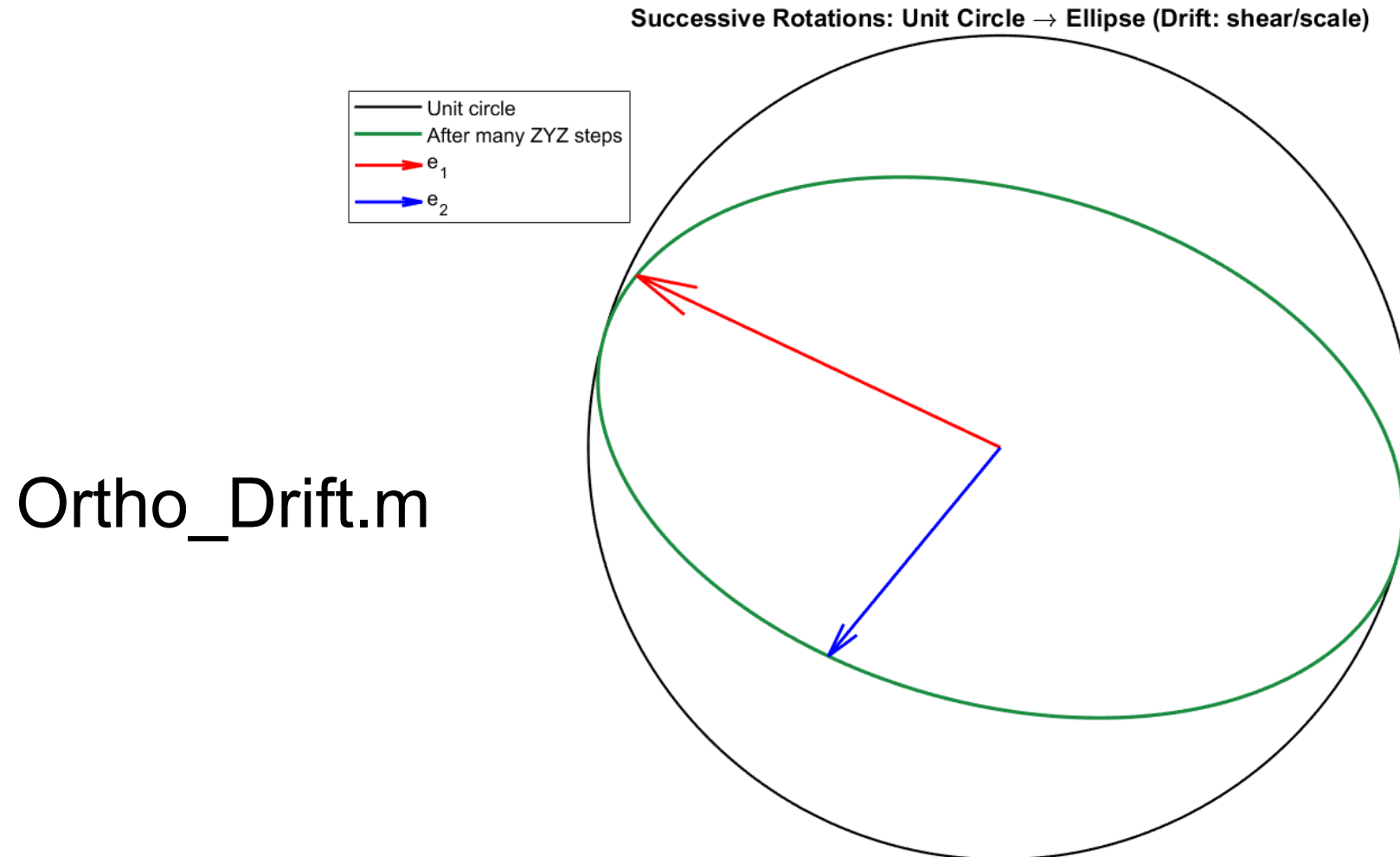
$$\| R_k^\top R_k - I \|_F = \sqrt{\sum_{i,j} (R_k^\top R_k - I)_{ij}^2}$$

- grows as columns lose unit length and perpendicularity → **shear/scale** creep in.
- With thousands of steps in **single precision**, both metrics drift:

$$\begin{aligned} \det(R_k) &\neq 1 \\ \| R_k^\top R_k - I \|_F &> 0 \end{aligned}$$



Example: Orthogonality Drift





Examples of Rotation Drift Problems

IMU Orientation Integration (Inertial Navigation)

- Robots, drones, and mobile platforms integrate gyroscope readings to update their orientation:

$$R_{k+1} = R_k R(\Delta\theta_k)$$

- Because this update happens **hundreds of times per second**, small floating-point errors in R_k accumulate.
 - $\det(R_k) \neq 1$: artificial **scale** appears
 - $R_k^T R_k \neq I$: axes lose **orthogonality**
 - orientation estimate slowly becomes **invalid**
 - robot's pose “leans,” even when stationary
- This is why IMU filters (e.g. **EKF**) often use **quaternions** and periodic **re-orthonormalization**.



Examples of Rotation Drift Problems

Forward Kinematics of Robotic Manipulators

- A robot arm computes the end-effector pose by chaining joint transforms:

$$T = T_1 T_2 T_3 \cdots T_n$$

- Each joint rotation contributes a rotation block. With many joints (e.g., 6-DOF industrial arm), repeated multiplication leads to:
 - orthogonality drift $\rightarrow$ end-effector frame tilts
 - accumulated errors in $R \rightarrow$ errors in **position** (via $p' = Rp$)
 - inconsistencies in Jacobians and controllers
- Industrial robot controllers periodically **re-orthonormalize** the rotation matrices to stay inside the special orthogonal group $SO(3)$.



Examples of Rotation Drift Problems

Visual Odometry / SLAM Pose Estimation

- In odometry and SLAM, the robot updates its pose like:

$$T_{k+1} = T_k \Delta T_k$$

- where ΔT_k comes from feature tracking or depth estimation.
If rotation matrices accumulate drift:
 - camera orientation becomes inaccurate
- This is why SLAM systems often use:
 - quaternions
 - orthonormalization after each pose integration step



Examples of Rotation Drift Problems

Flight Controllers on Quadrotors

- Quadrotors frequently update attitude:

$$R_{k+1} = R_k R(\omega_k \Delta t)$$

- At high update rates (up to 1 kHz), drift in R :
 - causes incorrect thrust vectoring
 - leads to unstable control
 - produces artificial yaw/pitch/roll bias



Examples of Rotation Drift Problems

Mobile Robots Using Dead Reckoning

- For wheeled robots using odometry:

$$R_{k+1} = R_k R(\Delta\theta)$$

- After thousands of updates:
 - heading drifts
 - path appears curved when the robot moved straight
 - localization errors grow
 - Though wheel slip also contributes, *numerical drift* plays a role when using rotation matrices.



Introduction to Quaternions



Rotation in 2D Using Imaginary Numbers

- A 2D point (x, y) can be represented as a **complex number**

$$p_0 = x_0 + iy_0.$$

- Or in the polar form

$$p_0 = r_0 e^{i\phi}$$

Where

$$r_0 = \sqrt{x_0^2 + y_0^2}, \quad \phi = \tan^{-1} \frac{y_0}{x_0}$$

- In the complex plane, a rotation by an angle θ about the origin is performed simply by **multiplying** the complex number by the **complex exponential**

$$e^{i\theta} = \cos \theta + i \sin \theta.$$



Rotation in 2D Using Imaginary Numbers

- Rotation by angle θ is equivalent to

$$p_1 = e^{i\theta} p_0 = r_0 e^{i(\phi+\theta)}, \quad \text{or} \quad p_1 = (\cos\theta + i\sin\theta)(x_0 + iy_0).$$

- which yields

$$x_1 = x_0 \cos\theta - y_0 \sin\theta,$$

$$y_1 = x_0 \sin\theta + y_0 \cos\theta.$$

- These are the same equations as the standard **2D rotation matrix**:

$$\begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}.$$

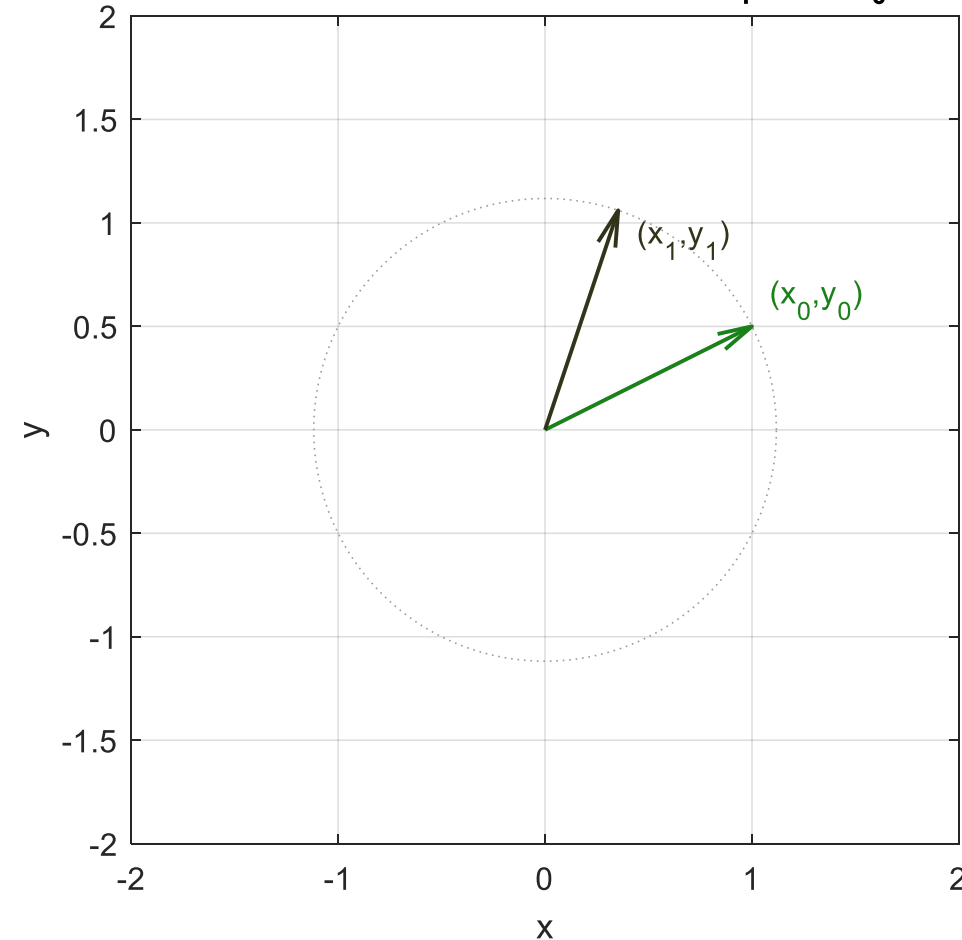
- Thus, complex multiplication performs a 2D rotation **without** the need for matrices.



Rotation in 2D Using Imaginary Numbers

$$\begin{aligned} p_1 &= e^{\frac{\pi}{4}} p_0 \\ &= 0.3536 + 1.0607ip_0 \\ &= 1 + 0.5i \end{aligned}$$

2D Rotation via Complex Multiplication: $p_1 = e^{i\theta} p_0$ ($\theta = 45^\circ$)





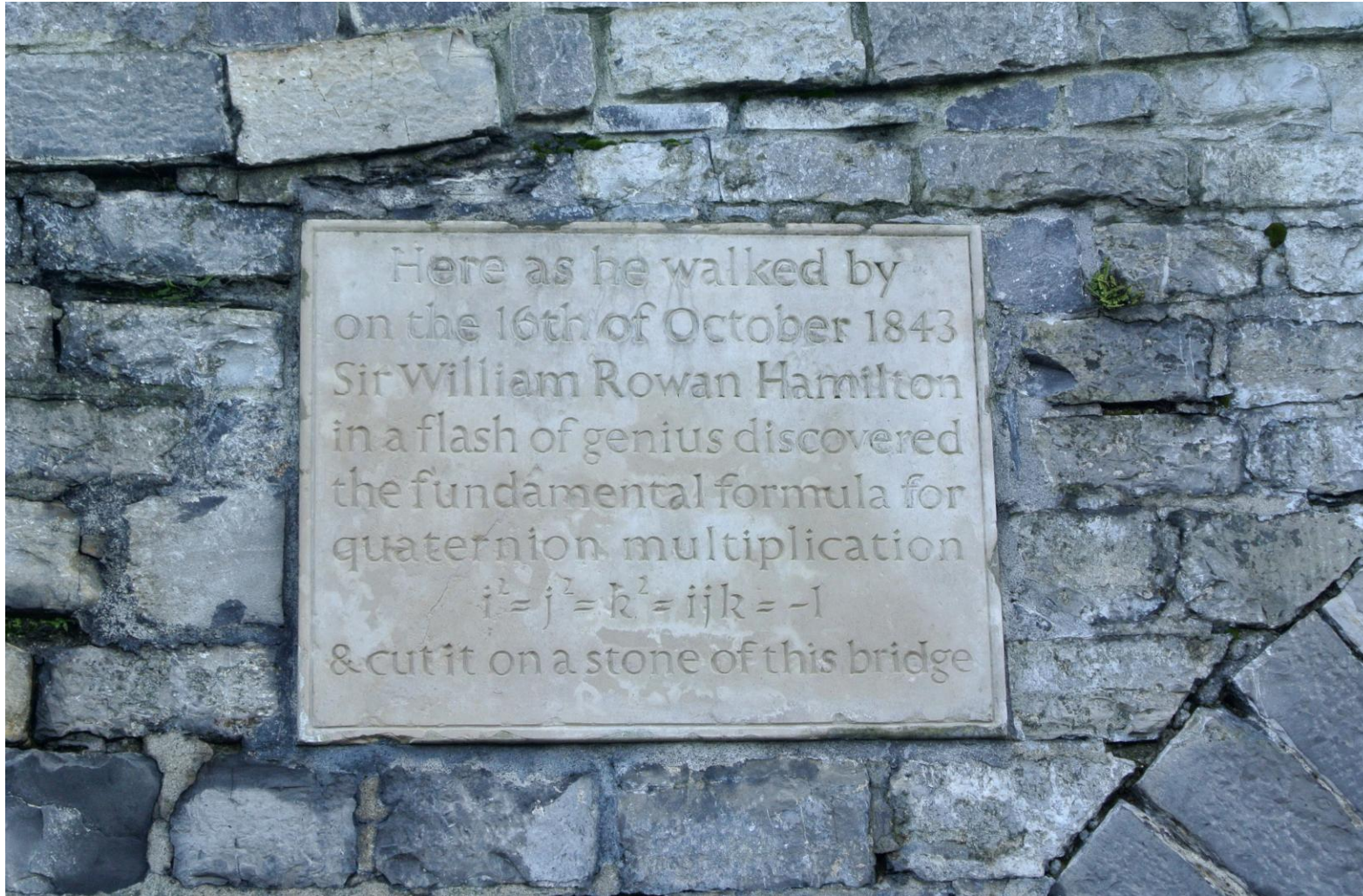
Quaternions

Hamilton's Challenge

- How do we extend the notion of a complex number so that it can represent a **3D rotation** using multiplication?
- Can this be done using a **three-dimensional complex number**?
- Hamilton discovered—while walking on the Broom Bridge—that the answer was **no**.
- He realized he needed a **four-dimensional hypercomplex number**, which he named the **quaternion**.



Quaternions





Quaternions

- In **3D**, rotation cannot be represented with a single imaginary number.
- Quaternions extend complex numbers by introducing **three imaginary units** i , j , k ,
- with multiplication rules:

$$\begin{aligned}i^2 &= j^2 = k^2 = ijk = -1, \\ij &= k, jk = i, ki = j, \\ji &= -k, kj = -i, ik = -j.\end{aligned}$$



Quaternions

- A quaternion Q is defined to have the form:

$$Q = q_0 + q_1\mathbf{i} + q_2\mathbf{j} + q_3\mathbf{k} \quad q_i \in \mathbb{R}, \mathbf{i} = 0,2,3$$

Where q_0 is the scalar component of Q and $\vec{q} = (q_1, q_2, q_3)$ is the vector component.

- Shorthand notation is $Q = (q_0, \vec{q})$ with $q_0 \in \mathbb{R}$ and $\vec{q} \in \mathbb{R}^3$.
- The magnitude of a quaternion satisfies

$$\|Q\|^2 = Q \cdot Q^* = q_0^2 + q_1^2 + q_2^2 + q_3^2$$



Quaternions

- The conjugate of a quaternion $Q = (q_0, \vec{q})$ is given by

$$Q^* = (q_0, -\vec{q})$$

- The inverse of quaternion is

$$Q^{-1} = \frac{Q^*}{\|Q\|^2}$$

- The product between two quaternion

$$Q.P = (q_0p_0 - \vec{q} \cdot \vec{p}, q_0\vec{p} + p_0\vec{q} + \vec{q} \times \vec{p})$$

- The identity element for quaternion multiplication

$$Q = (1, 0)$$



Quaternions

- The unit quaternions are the subset of all quaternion such that

$$\|Q\| = 1$$

- Given an angle axis representation of a rotation $k = (k_x, k_y, k_z)$ and angle θ , we define the associated unit quaternion as

$$Q = \left(\cos\left(\frac{\theta}{2}\right), k \sin\left(\frac{\theta}{2}\right) \right)$$

- Inversely, we can extract the corresponding rotation by setting

$$\theta = 2 \cos^{-1} q_0, \quad k = \begin{cases} \frac{\vec{q}}{\sin(\theta/2)}, & \text{if } \theta \neq 0 \\ 0, & \text{otherwise} \end{cases}$$



Quaternions

- Quaternions can be used to rotate vector p as follows:

$$p' = QpQ^*$$

- Note: The point can be treated as a vector, which is equivalent to a quaternion with a zero-scalar component $P = (0, p)$.



Quaternions

- Example: $v^0 = (0, 1, 1)$ is rotated about y_0 by $\frac{\pi}{2}$

$$Q = \left(\cos\left(\frac{\pi}{4}\right), 0, \sin\left(\frac{\pi}{4}\right), 0 \right),$$

$$Q^* = \left(\cos\left(\frac{\pi}{4}\right), 0, -\sin\left(\frac{\pi}{4}\right), 0 \right)$$

- For more illustration of unit quaternion as a rotation operator refer to [Visualizing quaternions, an explorable video series \(eater.net\)](#)



Quaternions

- **Euler angles** describe rotation using **three angles** (ϕ, θ, ψ).
- This parameterization is **not global**: there are orientations where Euler angles cannot represent rotation smoothly or uniquely.
- When the middle angle equals $\pm 90^\circ$, two of the rotation axes **align**, reducing 3D movement to effectively **2 dimensions**.
- This loss of a degree of freedom is called **gimbal lock**.
- The system can no longer rotate independently around all three axes.
- Small intended changes cause large and unpredictable rotations.
- Motion becomes unstable or uncontrollable.



Quaternions

- A 3D rotation matrix is never singular.
- Euler-angle *parameterizations* can become singular (gimbal lock) for certain orientations.
- Each Euler sequence (ZYZ, ZYX, XYX, ZXZ, etc.) has **different singularity conditions**.
- Therefore, a rotation matrix that is singular in the **ZYZ** convention may be **non-singular** in **XYX** or other sequences.
- Extracting Euler angles in a different sequence can succeed even if another sequence fails.



Quaternions

- **Quaternions**, however, represent rotation with **four numbers** (q_0, q_1, q_2, q_3) that follow the constraint
$$q_0^2 + q_1^2 + q_2^2 + q_3^2 = 1.$$
- These four components give a “global” description in the sense that **every possible 3D orientation** is represented smoothly and continuously in quaternion space.
 - Unique and smooth interpolation between orientations (important for robotics, graphics, animation).
 - Works consistently across all 3D orientations.



Quaternions

- Quaternions do **not use sequences of axis rotations**.
- Rotations are encoded as a single unit quaternion:

$$Q = \cos \frac{\theta}{2} + \sin \frac{\theta}{2} (p_1 i + p_2 j + p_3 k).$$

- Since quaternion representation does not depend on the order of rotations, no two axes ever "line up" and no singularity forms.



Quaternions Advantages over Matrices

- Compact (4 numbers as opposed to 9 for 3x3 rotation matrices)
- Efficient to compose (16 multiplications as opposed to 27 for matrices)
- Easy normalization after several compositions – handle numerical inaccuracies arising out of floating-point calculations easily
- Concatenate and interpolate nicely



Motion Interpolation

Linear Interpolation of Rotation Matrices

- Given two rotation matrices R_0 and R_1 :
$$R(t) = (1 - t)R_0 + tR_1, \quad 0 \leq t \leq 1$$
- Matrix interpolation is *not* constrained to the rotation group
- Thus $R(t)$ generally becomes:
 - non-orthogonal
 - with $\det(R(t)) \neq 1$
 - violating rigid-body motion (introduces scale/shear)
- So *linear interpolation of matrices is not suitable for smooth rotation.*



Motion Interpolation

Linear Interpolation of Quaternions

$$Q(t) = (1 - t)Q_0 + tQ_1$$

- After normalization:

$$\hat{Q}(t) = \frac{Q(t)}{\|Q(t)\|} = \frac{(1 - t)Q_0 + tQ_1}{\sqrt{q_0^2 + q_1^2 + q_2^2 + q_3^2}}$$

- Uniform values of t do **not** give uniform rotation speed because linear interpolation does not move along the unit sphere in quaternion space.



Motion Interpolation

Spherical Linear Interpolation (SLERP)

- SLERP moves exactly along the **shortest arc on the unit quaternion hypersphere**, producing constant-speed, smooth rotation.
- For angle ϕ defined by

$$\cos \phi = Q_0 \cdot Q_1$$

- The spherical interpolation is:

$$Q(t) = \frac{\sin((1-t)\phi)}{\sin \phi} Q_0 + \frac{\sin(t\phi)}{\sin \phi} Q_1$$

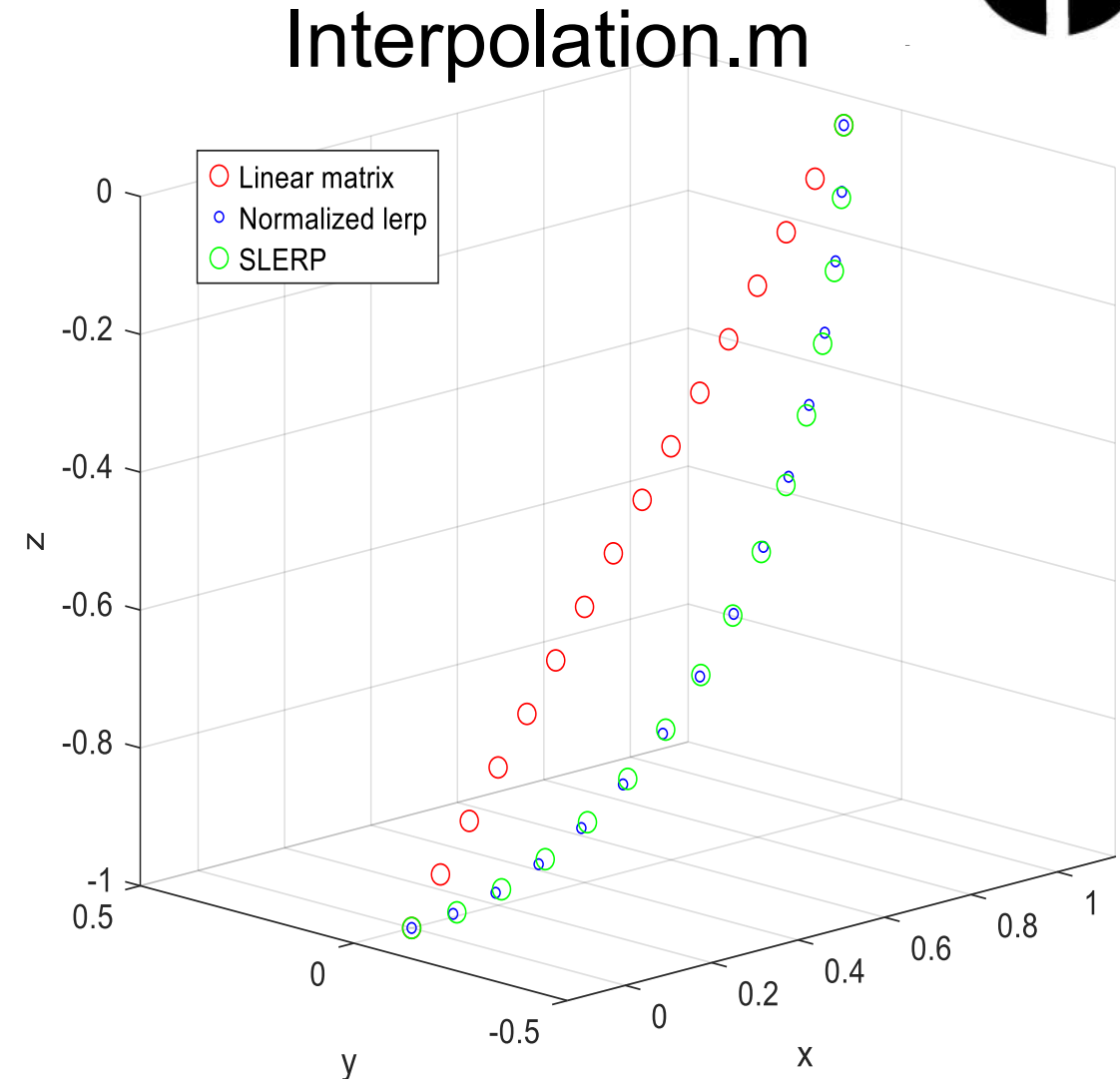
- **Properties of SLERP**

- Always stays on the unit quaternion sphere
- Produces constant-speed rotational motion
- Ensures rigid-body rotation with no distortion



Motion Interpolation

- **Linear Matrix Interpolation**
 - Red points do *not* lie on a sphere → rigid-body violation
 - The “rotation” includes scaling/skew
- **Normalized Quaternion LERP**
 - Blue points lie on the sphere (good)
 - Uneven spacing → non-uniform motion
- **SLERP**
 - Green points lie evenly spaced on the sphere
 - True constant-speed rigid-body rotation





Any Questions